

L29: 2^k Factorial Design for Simulation

Simulation Without a Black Box

Outline

From Scenarios to Factors

The 2^2 Design

Main Effects and Interactions

2^3 Yates Algorithm

Prediction Equation

Fractional Factorial

Key Takeaways

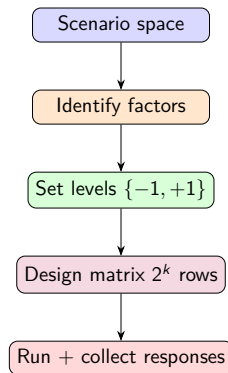
From Scenarios to Factors: The Vocabulary

Key Terms

- **Factor:** a controllable input we deliberately vary (e.g., number of servers c , service discipline)
- **Level:** a specific value of a factor (e.g., $c \in \{1, 2\}$ = two levels)
- **Treatment:** a combination of factor levels (one row of the design matrix)
- **Response:** the output KPI we measure (e.g., W_q , throughput, utilization)

Why design experiments?

- Ad-hoc “try everything” runs waste simulation budget
- Factorial design lets us estimate *all main effects* and *interactions* with the minimum number of runs

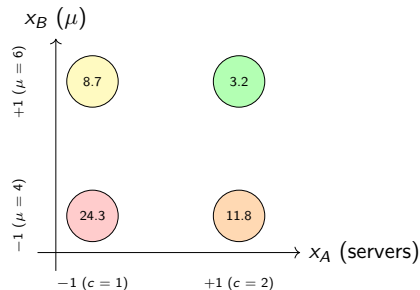


The 2² Design: Four Treatments

Clinic example:

- Factor A: number of servers (c): $\{1, 2\}$ coded $\{-1, +1\}$
- Factor B: service rate (μ): $\{4, 6\} \text{ min}^{-1}$ coded $\{-1, +1\}$
- Response: W_q (mean wait in queue, minutes)

x_A	x_B	Treatment	$\bar{W}_q$
-1	-1	(1)	24.3
+1	-1	<i>a</i>	11.8
-1	+1	<i>b</i>	8.7
+1	+1	<i>ab</i>	3.2



Each cell is the mean over $r = 30$ replications.

Main Effects and Interaction Calculation

Using the coded 2² responses (denote: (1) = 24.3, $a = 11.8$, $b = 8.7$, $ab = 3.2$):

Main effect of A (servers):

$$ME_A = \frac{(a - (1)) + (ab - b)}{2} = \frac{(11.8 - 24.3) + (3.2 - 8.7)}{2} = -9.0$$

Adding a server *reduces* W_q by 9.0 min on average.

Main effect of B (μ):

$$ME_B = \frac{(b - (1)) + (ab - a)}{2} = \frac{(8.7 - 24.3) + (3.2 - 11.8)}{2} = -12.1$$

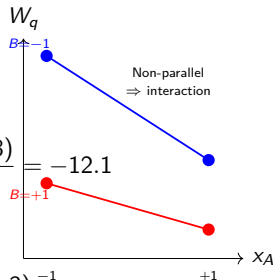
Interaction AB:

$$INT_{AB} = \frac{(ab - b) - (a - (1))}{2} = \frac{(3.2 - 8.7) - (11.8 - 24.3)}{2} = +4.4$$

Interaction Plot

Lines that are *not parallel* signal an

interaction.



2^3 Design: Yates Algorithm

With three factors A, B, C: $2^3 = 8$ treatments. The **Yates algorithm** computes all 7 effects in a single systematic table pass.

Standard order: (1), a, b, ab, c, ac, bc, abc

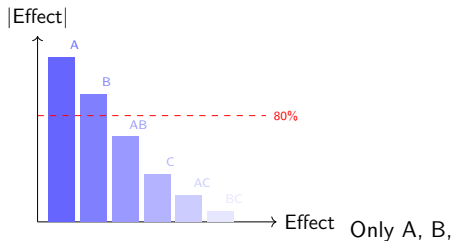
Procedure:

1. Write responses in standard order
2. Apply two passes: alternate sum/difference of adjacent pairs
3. Divide column 2 entries by $n \cdot 2^{k-1}$ to get effects

```
import numpy as np
```

```
Y = np.array([y1,y2,y3,y4,y5,y6,y7,y8])
for _ in range(3):           # k=3 passes
    Y = np.concatenate([
        Y[::2] + Y[1::2],
        Y[1::2] - Y[::2]])
effects = Y / (n * 4)        #  $2^{(k-1)}$ 
```

Pareto chart of |effects|:



Only A, B, and AB exceed the 80% threshold — the rest are negligible.

Linear Prediction Equation

Fitted Response Surface (coded units)

$$\hat{Y} = \mu + \alpha x_A + \beta x_B + \gamma x_A x_B$$

where μ = grand mean, $\alpha = ME_A/2$, $\beta = ME_B/2$,
 $\gamma = INT_{AB}/2$.

For our clinic 2² example:

$$\hat{W}_q = 12.0 - 4.5 x_A - 6.05 x_B + 2.2 x_A x_B$$

Prediction at $c = 2$, $\mu = 6$ (i.e., $x_A = +1$, $x_B = +1$):

$$\hat{W}_q = 12.0 - 4.5(1) - 6.05(1) + 2.2(1)(1) = 3.65 \approx 3.2\checkmark$$

Use for: quick interpolation between design points;
identifying the direction of steepest descent for optimization.

Limitation

The linear prediction is only valid *within the range of the design levels*. Extrapolating beyond $\{-1, +1\}$ violates the model and can produce absurd predictions (e.g., negative wait times).

Fractional Factorial Preview

With $k = 6$ factors: $2^6 = 64$ runs. A 2^{6-1} half-fraction uses only 32 runs and estimates all main effects and two-factor interactions — sufficient for screening.

Fractional Factorial: Screening with 2^{k-1}

Why fractional?

- 2^k grows exponentially: $k = 8 \Rightarrow 256$ runs
- In screening, we expect few *active* factors (effect sparsity)
- A half-fraction 2^{k-1} halves the runs at the cost of *aliasing* high-order interactions

Aliasing (confounding): In a 2^{3-1} design (4 runs), the interaction AB is aliased with factor C. If we observe a large AB estimate, it might be C.

Resolution III, IV, V

- **III:** MEs aliased with 2FIs — use only for screening
- **IV:** MEs clear; 2FIs aliased with each other
- **V:** MEs and 2FIs clear — generally preferred

```
import numpy as np

# 2^(3-1) half-fraction design
# Generator: C = AB
x = np.array([
    [-1, -1, 1], # C = AB = (-1)(-1) = +1
    [ 1, -1, -1], # C = (+1)(-1) = -1
    [-1, 1, -1], # C = (-1)(+1) = -1
    [ 1, 1, 1], # C = (+1)(+1) = +1
])
# Columns: [A, B, C]
# Only 4 runs for 3 factors!
```

After screening: identify active factors, then run a full 2^k or response-surface design on them only.

Key Takeaways

1. **Vocabulary:** factor, level, treatment, response. Coding $\{-1, +1\}$ simplifies effect calculations.
2. **2^2 design:** four treatments; directly estimates ME_A , ME_B , and INT_{AB} .
3. **Interaction plot:** non-parallel lines signal a 2FI; parallel lines mean the effects are additive.
4. **Yates algorithm:** systematic pass to extract all $2^k - 1$ effects without matrix inversion.
5. **Prediction equation:** $\hat{Y} = \mu + \alpha x_A + \beta x_B + \gamma x_A x_B$ — valid only within design bounds.
6. **Fractional factorial:** 2^{k-1} halves the runs; use Resolution V or higher when 2FIs matter.

Next Lecture

L30 — Sensitivity analysis: OAT, tornado diagrams, and Morris screening.